30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 23

CI/CD & DevOps — Part 1: Pipelines, Caching, Quality Gates & Device Tests



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23

Why CI/CD matters in 2026

A senior Android engineer is expected to own the path from commit to release, not only the code. Interviewers probe whether your pipeline is **fast, trustworthy and secure**:

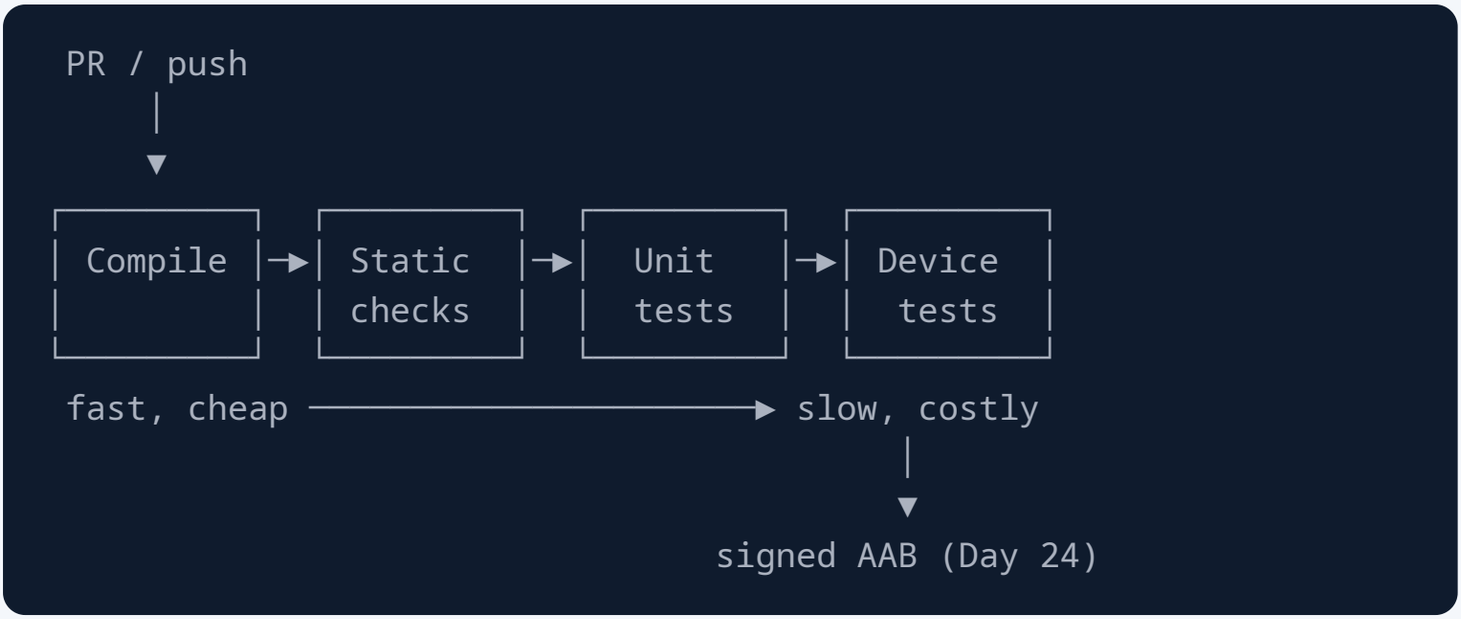
- Every merge to `main` should be releasable — the pipeline decides, not a person
- Gradle builds are slow: **caching** and **sharding** make CI usable
- Static analysis as a **gate** (Lint, detekt, ktlint), not a report nobody reads
- Real device tests with **Gradle Managed Devices** or **Firebase Test Lab**
- The pipeline itself is an **attack surface** (supply-chain security)

Level	Questions
Junior	CI vs CD · A first workflow · The PR gate
Mid	Gradle caching · Static analysis · Device tests in CI · Sharding
Senior	Pipeline per trigger · Flaky tests · Which gates block a merge
Architect	Portable pipelines & runners · Supply-chain security · Fixing a slow CI



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: cheap checks first



- **Fail fast:** the cheapest, most likely failures run first
- **Parallelize:** independent jobs (lint, unit tests) run side by side
- **Same tasks everywhere:** CI calls the Gradle tasks a developer can run locally

THE ONE IDEA BEHIND EVERY PIPELINE

A pipeline is a **feedback system**. Judge every step by two questions: *how fast does it tell me something is wrong, and can I trust it when it's red?*



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23

Junior

CI vs CD · A first workflow · The PR gate



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. CI, Continuous Delivery, Continuous Deployment

Term	Means	On Android
Continuous Integration	Everyone merges small changes into `main` often; each change is built and tested automatically	PR checks + a build of every `main` commit
Continuous Delivery	Every green `main` build **can** be released; producing the signed artifact is automated, the release is a decision	AAB uploaded to an internal track automatically
Continuous Deployment	Every green build reaches users with no human step	Rare on mobile: store review, users update binaries, no rollback

INTERVIEW TRAP

"CI means a server automatically merges everyone's code into main." (last year's wording) — **No.** Developers integrate; CI **verifies** every integration. CI is a practice — small, frequent merges. A team with Jenkins and three-week branches has a build server, not CI.

INTERVIEW TRAP

"We do continuous deployment to Google Play." — Usually it's continuous **delivery** to a testing track, then a deliberate promotion. On mobile you can't take a release back (Day 24).



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. A first GitHub Actions workflow

```yaml
name: ci
on:
  pull_request:
  push:
    branches: [main]
jobs:
  check:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17
      - uses: gradle/actions/setup-gradle@v4
      - run: ./gradlew assembleDebug testDebugUnitTest lintDebug
```

- `setup-gradle` caches Gradle User Home and provides build insights — no hand-made cache keys
- JDK 17 is what current AGP versions require; always build with the **wrapper** (`./gradlew`)
- File lives in `.github/workflows/ci.yml`; tag names shown are major versions — check for newer ones



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

INTERVIEW TRAP

"`./gradlew test` runs the unit tests." — It runs them for **every variant**: debug, release and each flavor. On a PR, `testDebugUnitTest` gives the same signal for a fraction of the time.

INTERVIEW TRAP

Last year's deck: *"`connectedCheck` runs all instrumented tests and lint checks."* — **Wrong.** `connectedCheck` runs instrumented tests on connected devices only. Lint is its own task (`lintDebug`).

FOLLOW-UP

"Why set `timeout-minutes`?" — The default job timeout on GitHub Actions is 6 hours. One hung emulator or deadlocked test burns 6 hours of runner time before anyone notices.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What runs on every pull request

The **PR gate** — required status checks in branch protection (or rulesets). Nothing merges red.

- **Compile** the variants you ship (at least debug on PRs)
- **Unit tests** for the changed code
- **Lint, detekt, ktlint** — style and bug patterns stop at the door
- **Architecture tests** (Konsist, Day 3) — cheap and deterministic
- On `main` (or nightly): a **release build with R8**, because shrinking and obfuscation bugs only show up there

INTERVIEW TRAP

"Run everything on every PR." — A full emulator matrix on each push makes feedback slow, so developers batch changes into big PRs. That is the opposite of CI. Keep PR checks to minutes; move expensive suites to merge or nightly.

FEEDBACK TIME IS A FEATURE

If CI is slow, people stop waiting for it. A fast, reliable red build is worth more than a thorough one nobody trusts.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23

Mid-Level

Gradle caching · Static analysis · Device tests in CI · Sharding



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Gradle caching in CI

Layer	What it saves	How
Gradle User Home	Downloaded dependencies, wrapper distribution	`setup-gradle` on GitHub; on GitLab, `GRADLE_USER_HOME` inside the project + `cache:`
Build cache	Task outputs (compile, dex, tests) reused across builds and machines	`org.gradle.caching=true` + a remote cache node or Develocity
Configuration cache	The configured task graph — skips configuration	`org.gradle.configuration-cache=true`; `setup-gradle` saves it only with an encryption key

- By default, `setup-gradle` **writes** the cache only from the default branch; PR builds read it
- Local and remote build caches reuse outputs only for **cacheable, reproducible** tasks



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. A remote build cache — and its traps

```kotlin
// settings.gradle.kts
val isCi = providers.environmentVariable("CI").isPresent

buildCache {
    remote<HttpBuildCache> {
        url = uri("https://gradle-cache.example.com/cache/")
        isPush = isCi
        credentials {
            username = providers
                .environmentVariable("CACHE_USER").orNull
            password = providers
                .environmentVariable("CACHE_PASSWORD").orNull
        }
    }
}
```

INTERVIEW TRAP

Letting developer machines push to the remote cache. One misconfigured laptop can poison it for everyone. **CI pushes, laptops only pull.**

INTERVIEW TRAP

A `buildConfigField` with the build time or git hash in every debug build changes the inputs of every build — cache hits drop to zero. Keep such values out of debug builds.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Static analysis as a gate

Tool	Catches	Gradle task
Android Lint	Android-specific bugs: `NewApi`, missing permissions, unused resources, accessibility, security	`lintDebug`
detekt	Kotlin code smells, complexity, likely bugs; custom rules	`detekt`
ktlint	Formatting and the official Kotlin style	`ktlintCheck` (ktlint-gradle) or `spotlessCheck`
Compose rules	Compose API misuse: modifier parameters, state hoisting, naming	compose-rules for ktlint/detekt, or Slack's compose-lints

INTERVIEW TRAP

Last year's deck ran `./gradlew detektCheck` — **that task doesn't exist.** The detekt plugin's task is `detekt` (plus type-resolution variants such as `detektMain`).

LET THE FORMATTER WIN

Run `ktlintFormat` (or Spotless) before committing, so code review is about design, not spaces.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Configure the gates

```
// app/build.gradle.kts
android {
    lint {
        abortOnError = true
        checkDependencies = true
        baseline = file("lint-baseline.xml")
        sarifReport = true
    }
}

detekt {
    buildUponDefaultConfig = true
    config.setFrom("$rootDir/config/detekt.yml")
    baseline = file("detekt-baseline.xml")
}
```

- A **baseline** freezes today's issues; only **new** issues fail the build — the way to adopt a gate on a legacy codebase
- **SARIF** reports can be uploaded to GitHub code scanning, so findings show up on the PR diff

INTERVIEW TRAP

Regenerating the baseline "to make CI green". Now new issues are hidden too. A baseline should **only shrink** — track its size like debt.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Instrumented tests in CI: three options

Option	Runs on	Good for	Watch out
Emulator on the runner	Linux runner with KVM enabled	Small suites, full control	Boot time, runner CPU, flakiness
Gradle Managed Devices	Emulators that AGP creates and manages	Declared in Gradle; same setup locally and in CI	Still emulators; needs KVM
Firebase Test Lab	Google's physical and virtual devices	Real hardware, many API levels, sharding	Cost and quota, slower feedback

- Emulator on the runner: usually the `reactivecircus/android-emulator-runner` action

INTERVIEW TRAP

"Run every device test on every API level for every PR." — Pick one representative device for PRs. Run the matrix (API levels, form factors, real devices) on `main` or nightly.

FOLLOW-UP

"Why does KVM matter?" — x86 emulator images need hardware acceleration; without it the emulator refuses to start or is unusably slow. On GitHub's Linux runners you grant KVM access with a udev-rule step before booting it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Gradle Managed Devices

```
android {
    testOptions {
        animationsDisabled = true
        execution = "ANDROIDX_TEST_ORCHESTRATOR"
        managedDevices {
            localDevices {
                create("pixel6Api34") {
                    device = "Pixel 6"
                    apiLevel = 34
                    systemImageSource = "aosp-atd"
                }
            }
        }
    }
}
```

```
./gradlew pixel6Api34DebugAndroidTest
```

- **ATD** (Automated Test Device) images are slimmed-down system images for headless tests
- **Orchestrator** runs each test in its own instrumentation — one crash doesn't kill the suite (add `androidTestUtil` for `androidx.test:orchestrator`)

INTERVIEW TRAP

ATD images remove some system apps and services. A test that needs them fails only on ATD — switch that suite to a regular `aosp` or `google` image.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Test sharding: split the wall-clock time

- Sharding cuts **wall-clock** time, not machine time: every shard pays for checkout, Gradle setup and emulator boot
- **Unit tests:** parallel forks inside one job first; split by module across jobs only in big repositories
- **Device tests:** AndroidJUnitRunner shards natively with `numShards` and `shardIndex`

```
// build-logic convention plugin
tasks.withType<Test>().configureEach {
    maxParallelForks =
        (Runtime.getRuntime().availableProcessors() / 2)
            .coerceAtLeast(1)
}
```

INTERVIEW TRAP

"Uniform shards are balanced." — Uniform sharding splits by **test count**, not duration. One slow class makes one shard the bottleneck. Timing-based sharding (e.g., Flank) balances by past run times.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Sharding device tests across machines

```yaml
device-tests:
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false
    matrix:
      shard: [0, 1, 2, 3]
  steps:
    # checkout, JDK, setup-gradle, enable KVM …
    - run: ./scripts/device-tests.sh ${{ matrix.shard }} 4
```

```bash
# scripts/device-tests.sh <shardIndex> <numShards>
ARGS=-Pandroid.testInstrumentationRunnerArguments
./gradlew pixel6Api34DebugAndroidTest \
  "$ARGS.shardIndex=$1" "$ARGS.numShards=$2"
```

- One big machine instead: GMD can shard across several emulators with `android.experimental.androidTest.numManagedDeviceShards`
- Firebase Test Lab: the `--num-uniform-shards` flag of `gcloud firebase test android run`
- `fail-fast: false` — let every shard finish, so you see all failures at once



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23

Senior

Pipeline per trigger · Flaky tests · Which gates block a merge



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Design the pipeline per trigger

Trigger	Runs	Goal
Pull request	Compile, unit tests, Lint/detekt/ktlint — affected modules	Feedback in minutes
Merge queue (`merge_group`)	Same checks on the PR **combined with latest `main`**	Two green PRs can't break `main` together
Push to `main`	Full unit suite, R8 release build, smoke device tests, upload to internal testing	`main` is always releasable
Nightly (`schedule`)	Device matrix on Firebase Test Lab, screenshot tests, benchmarks	Expensive coverage
Tag / release branch	Signed AAB, release notes, Play upload	Day 24

FOLLOW-UP

"Why a merge queue?" — Two PRs can each pass against an old `main` and still conflict semantically (one renames a function the other calls). The queue tests them in merge order.



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Triggers and concurrency in YAML

```yaml
on:
  pull_request:
  merge_group:
  push:
    branches: [main]
  schedule:
    - cron: "0 2 * * *"   # nightly, UTC
  workflow_dispatch:

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}
```

- A new push to a PR **cancels** the outdated run; runs on `main` are never cancelled — each commit is a release candidate

INTERVIEW TRAP

A `paths:` filter on a workflow that provides a **required** check. When the workflow is skipped, the check stays *pending* and the PR can't merge. Filter inside the job instead, or add a final "gate" job that always reports.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Flaky tests: keep red meaningful

A flaky test passes and fails on the **same code**. The real cost: people rerun until green, then start ignoring real failures.

Step	How
Detect	Track results per test across runs (JUnit XML history, Develocity, Test Lab results)
Contain	Retry a limited number of times — and **report** "passed on retry" as flaky
Quarantine	Move it to a non-blocking suite, with an owner and a deadline
Fix	Shared state, real time or network, animations, missing idling resources, test order

```
// plugins { id("org.gradle.test-retry") }
tasks.withType<Test>().configureEach {
    retry {
        maxRetries.set(2)
        failOnPassedAfterRetry.set(false)
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What interviewers check

- Device tests: Orchestrator with `clearPackageData` isolates state between tests
- Firebase Test Lab can rerun failures with `--num-flaky-test-attempts`
- Watch the **flaky rate** as a CI health metric (Q13)

INTERVIEW TRAP

"Just add retries." — Silent retries hide real bugs: a race condition in a test is often a race condition in production. Retries must be visible, counted and owned.

INTERVIEW TRAP

`@Ignore` without an owner or ticket. An ignored test is a deleted test that still costs compile time.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Which gates should block a merge?

Gate	Block the PR?	Why
Compile, unit tests, Lint errors, detekt, ktlint	**Yes**	Cheap and deterministic
Architecture tests (Konsist, Day 3)	**Yes**	Protects module and layer boundaries
Screenshot tests (Roborazzi/Paparazzi, Day 10)	**Yes**, with reviewed image diffs	Run on the JVM — fast
Coverage	As a **signal** on changed lines	See the trap
App size	Warn when growth exceeds a budget	Needs a release build
Benchmarks	Nightly with regression detection (Day 28)	Too noisy on shared runners
Dependency updates	Renovate/Dependabot PRs pass the same gate	Small, frequent upgrades



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. The coverage trap

INTERVIEW TRAP

Last year: *"Fail the build if coverage drops below 75% — this ensures new code is always accompanied by tests."* — Coverage measures lines **executed**, not behavior **verified**. A test without assertions can reach 100%. A global threshold rewards junk tests and punishes deleting dead code.

- Better: **diff coverage** on changed lines as a PR comment, plus reviewer judgment
- Tools: **Kover** (`koverVerify`), or AGP's built-in JaCoCo: `enableUnitTestCoverage = true` on a build type, then `createDebugUnitTestCoverageReport`
- `jacocoTestReport` comes from Gradle's `jacoco` plugin for JVM modules — AGP doesn't create it for Android modules

FOLLOW-UP

"So is coverage useless?" — No. **Low** coverage in critical code is a real warning. **High** coverage proves nothing by itself.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23

Architect

Portable pipelines & runners · Supply-chain security · Fixing a slow CI



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Keep logic in Gradle, keep YAML thin

The same Gradle tasks, a different CI:

```yaml
# .gitlab-ci.yml
variables:
  GRADLE_USER_HOME: "$CI_PROJECT_DIR/.gradle"
cache:
  key:
    files: [gradle/wrapper/gradle-wrapper.properties]
  paths: [.gradle/caches/, .gradle/wrapper/]
check:
  image: $ANDROID_BUILD_IMAGE   # JDK 17 + Android SDK
  script:
    - ./gradlew assembleDebug testDebugUnitTest lintDebug
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH
```

- GitLab caches only paths **inside** the project directory — hence the `GRADLE_USER_HOME` move
- CI YAML only prepares the machine and calls Gradle. Build logic lives in convention plugins (Day 21), so developers can run it locally and you can switch CI vendors



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Choosing platforms and runners

Choice	Strength	Cost / risk
GitHub Actions, hosted	No ops, marketplace, merge queue	Per-minute cost; macOS minutes (iOS/KMP) cost more
GitLab CI	Code, MRs, CI and registry in one; mature self-managed runners	Cache must live in the project directory
Self-hosted / autoscaled runners	Big machines, warm caches, private network	You own patching and isolation
Mobile CI SaaS (Bitrise, Codemagic)	Preinstalled Android/iOS tooling	Lock-in through vendor-specific steps

INTERVIEW TRAP

Last year: *"A globally distributed remote cache is a game-changer."* — Only if tasks are cacheable and the cache is **close to the runners**. A cache on another continent can be slower than rebuilding. Measure hit rate and download time first.

INTERVIEW TRAP

Self-hosted runners on a **public** repository: any fork PR can run code on your machine and leave something behind. Use ephemeral runners, and never for untrusted PRs.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The pipeline is an attack surface

```yaml
permissions:
  contents: read      # least privilege by default

on:
  pull_request:       # fork PRs: no secrets, read-only token
# pull_request_target runs with secrets and a write token:
# never check out and build PR code under it
```

```
# Record checksums of every dependency and plugin;
# builds fail on mismatch. Review the file in PRs.
./gradlew --write-verification-metadata sha256 help
```

INTERVIEW TRAP

"Our repo is private, so CI security doesn't matter." — In March 2025 the popular `tj-actions/changed-files` action was compromised: every workflow using its tag ran code that printed secrets into build logs. The fix is to **pin third-party actions to a full commit SHA** and update pins through Dependabot.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. The supply-chain toolbox

Risk	Control
Compromised third-party action	Pin to a commit SHA; automated pin updates
Over-privileged `GITHUB_TOKEN`	`permissions:` read-only by default, write only per job
Secrets reaching untrusted code	`pull_request` for forks; release secrets in protected environments (Day 24)
Malicious Gradle wrapper jar	Wrapper validation (`gradle/actions/wrapper-validation`)
Tampered dependency	Gradle dependency verification; repository content filtering
Persistent self-hosted runner	Ephemeral runners, one job each

MY TAKE

Treat the pipeline like production code: reviewed, least-privileged, pinned and auditable. The release job holds your signing and Play credentials — it's the most valuable target in the company's Android stack.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. "Our CI takes 40 minutes. Fix it."

1. **Measure first.** Build scans (`--scan`, Develocity): where does time go — queue, setup, configuration, compilation, tests, emulator boot?
2. **Stop doing work.** Configuration cache, remote build cache, `testDebugUnitTest` instead of `test`, and only **affected modules** (e.g., Dropbox's Affected Module Detector).
3. **Parallelize.** Independent jobs side by side; shard device tests.
4. **Move work.** Expensive suites to the merge queue or nightly.
5. **Buy time.** Bigger runners often cost less than the engineers waiting for them.
6. **Fix the structure.** A monolithic `:app` module limits every cache — modularization is Day 25.

INTERVIEW TRAP

"Affected-only testing is always safe." — Only if the dependency graph is right. Changes to build logic, the version catalog or shared resources must trigger everything — and `main` still runs the full suite.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Metrics for a healthy CI

Metric	Tells you
PR feedback time (p50 / p90)	What developers actually feel
Queue time	A runner-capacity problem, not a build problem
Build cache hit rate	Whether caching really works
Flaky failure rate	Whether people can trust a red build
Time `main` is red	How often "always releasable" is false
Cost per PR	Input for budget decisions

SENIOR ANSWER IN ONE LINE

"CI is a product for developers: fast feedback on every PR, trustworthy red builds, expensive checks where they cost least — and a pipeline as locked down as the release it produces."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 23 · ARCHITECT

Day 23 — cheat sheet

- CI = small, frequent merges, each verified automatically. Mobile usually means continuous **delivery**, not deployment.
- Cheap checks first; one workflow calls the same Gradle tasks developers run locally.
- `testDebugUnitTest`, not `test`; `connectedCheck` runs device tests only — Lint is separate.
- Set `timeout-minutes`; cancel outdated PR runs, never `main` runs.
- Cache Gradle User Home with `setup-gradle`; remote build cache — CI pushes, laptops pull.
- Lint, detekt (`detekt`, not `detektCheck`), ktlint as required checks; baselines only shrink.
- Device tests: one device on PRs, matrix nightly — GMD with ATD images or Firebase Test Lab; shard by `numShards` / `shardIndex`.
- Flaky tests: visible retries, quarantine with an owner, fix the root cause.
- Coverage is a signal, not a gate. Merge queue protects `main`.
- Pin actions to SHAs, least-privilege tokens, no secrets for fork PRs, dependency verification.

Day 24 is next → CI/CD part 2: signing, releases & rollouts. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700